Comparativa de los algoritmos LGC y Mersenne Twister aplicados a simulaciones educativas

11 de diciembre de 2024

Pregunta de Investigación:
¿Cómo impactan las propiedades estadísticas del LCG y el Mersenne Twister en la precisión y eficiencia de simulaciones basadas en actividades educativas?

Número de palabras: 2687
Asignatura: Matemáticas

Índice

1. Introducción **3**

2. Métodos de Generación de Números Pseudoaleatorios en Computación **4**
2.1. Definición de Aleatoriedad 4
2.2. Algoritmos de Generación de Números Pseudoaleatorios 4
2.2.1. Generador Congruencial Lineal (LCG) 4
2.2.2. Mersenne Twister 4

3. Aplicación de Métodos de Generación de Números Pseudoaleatorios en Simulaciones **5**
3.1. Implementación en Software 5
3.1.1. Método de Kolmogorov-Smirnov 5

4. Estudio de Caso: Simulación de Actividades de la Caja de Varga **6**

5. El Problema de los Desarreglos y su Aplicación a La Caja de Varga **7**
5.1. Definición del Problema 8
5.2. Demostración de la fórmula por inducción 8
5.3. Cálculo de el número de desarreglos de n elementos 8
5.4. Aplicación: La Caja de Varga 9
5.5. Cálculo de Desarreglos Parciales en el caso n = 6 y sus probabilidades . . . 9
5.6. Simulación del juego en Pygame 10

6. Conclusiones **13**

Referencias **14**

Anexo **15**
A. Test de Kolmogórov- Smirnov 15
B. Código del Viaje del Taxista 15
C. Código del Método de Urnas 20
D. Código de simulación de desarreglos usando Marsenne Twister 23
E. Código de simulación de desarreglos usando LGC 27

1. Introducción

Hoy en día, la producción de números aleatorios en ordenadores es especialmente significativa en el campo de las matemáticas y la computación. Aunque los ordenadores no logran generar auténtica aleatoriedad, se utilizan algoritmos deterministas para producir secuencias numéricas que buscan replicarla, conocidos como generadores números pseudoaleatorios.

La Caja de Varga es un recurso didáctico desarrollado por Tamás Varga, un matemático húngaro, a finales de la década de 1970. Su objetivo era desarrollar actividades para introducir conceptos educativos en probabilidad, combinatoria y estadística.

Pregunta de investigación: ¿Cómo impactan las propiedades estadísticas del LCG y el Mersenne Twister en la precisión y eficiencia de simulaciones basadas en actividades educativas?

El objetivo principal de este ensayo es analizar dos generadores de números pseudoaleatorios comunmente utilizados: el Generador Congruencial Lineal (LCG) y el Mersenne Twister. Además se pretende utilizarlos para crear simulaciones digitales de materiales que forman parte de la Caja de Varga. También se incluirá una evaluación de estas simulaciones en términos de precisión matemática y accesibilidad para los usuarios.

2. Métodos de Generación de Números Pseudoaleatorios en Computación

2.1. Definición de Aleatoriedad

La aleatoriedad describe eventos o resultados que ocurren sin un patrón predecible. Matemáticamente, la aleatoriedad se define en términos de probabilidad. Andrey Kolmogorov formalizó este concepto en 1933: *"Un resultado se considera aleatorio si no se puede predecir con certeza y cada posible resultado ocurre con una probabilidad bien definida"*.

Aunque el concepto de aleatoriedad ha existido desde la antigüedad, su formalización matemática es relativamente moderna. Civilizaciones como Mesopotamia, Egipto y China usaban dados para juegos de azar, pero fue en el siglo XVI cuando Gerolamo Cardano introdujo los primeros estudios matemáticos sobre la probabilidad en juegos de azar. La correspondencia entre Blaise Pascal y Pierre de Fermat en el siglo XVII sentó las bases de la teoría de la probabilidad moderna. Desde entonces, avances importantes como la ley de los grandes números de Jakob Bernoulli y el teorema central del límite de Abraham de Moivre permitieron un análisis más riguroso de los fenómenos aleatorios (Chaparro, 2020).

2.2. Algoritmos de Generación de Números Pseudoaleatorios

Los **generadores de números pseudoaleatorios (PRNGs)** producen secuencias que imitan la aleatoriedad. Aunque son deterministas, su capacidad para aproximar la aleatoriedad los hace esenciales en simulaciones, criptografía y modelado de procesos estocásticos.

2.2.1. Generador Congruencial Lineal (LCG)

Uno de los algoritmos más antiguos y simples, definido por:

$$X_{n+1} = (aX_n + c) \mod m$$

donde: - a: Multiplicador. - c: Incremento. - m: Módulo. - X_0: Semilla inicial.

A pesar de su simplicidad, los LCG tienen limitaciones, como períodos cortos y patrones repetitivos, por lo que su uso es más adecuado en simulaciones básicas o juegos simples (Knuth,1969).

2.2.2. Mersenne Twister

Este algoritmo avanzado, desarrollado en 1997 por Matsumoto y Nishimura, tiene un período de $2^{19937} - 1$, garantizando una excelente uniformidad. Es preferido para simulaciones complejas por su largo período y alta calidad de aleatoriedad.

3. Aplicación de Métodos de Generación de Números Pseudoaleatorios en Simulaciones

A continuación, se estudiará cómo los generadores pseudoaleatorios son aplicados en simulaciones de actividades educativas, incluyendo su uso en el problema de los desarreglos.

3.1. Implementación en Software

El uso de Python para desarrollar esta simulación se fundamenta en su versatilidad y facilidad de uso. Es un lenguaje de programación que tiene una sintaxis clara, lo que facilita su comprensión, tanto para estudiantes como para educadores. Esta accesibilidad es fundamental en un entorno educativo, ya que permite a los usuarios sin experiencia en programación involucrarse en el desarrollo de simulaciones interactivas.

Una de las principales ventajas de Python es la disponibilidad de bibliotecas potentes, como random y numpy, que son esenciales para la generación de números pseudoaleatorios y la manipulación de datos.

3.1.1. Método de Kolmogorov-Smirnov

Para evaluar la calidad de los generadores de números pseudoaleatorios utilizados en este proyecto, se ha llevado a cabo un análisis estadístico a partir de la prueba de Kolmogorov-Smirnov para evaluar el comportamiento de los números generados por el Mersenne Twister y el Generador Congruencial Lineal (LCG).

La prueba de Kolmogorov-Smirnov es una herramienta estadística que se utiliza para comparar una muestra con una distribución de referencia, en este caso, la distribución uniforme. La prueba se basa en la diferencia máxima entre la función de distribución acumulativa (CDF) de los datos observados y la CDF de la distribución teórica esperada (Kolmogorov, 1933).

Se puede consultar el código del test en el apartado A del anexo.

Generador	**Estadística KS**	**p-valor**
LCG	0.0145	0.0303
Mersenne Twister	0.0086	0.4503

Cuadro 1: Resultados de la Prueba de Kolmogorov-Smirnov para LCG y Mersenne Twister

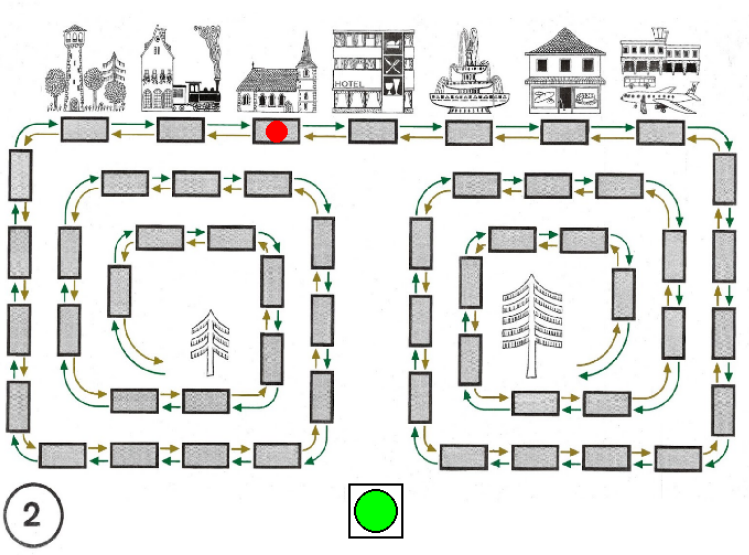
La estadística KS con un valor alto y el p-valor bajo indican que la secuencia generada por el LCG no es compatible con una distribución uniforme. Este resultado es consistente con las limitaciones teóricas del LCG, especialmente si no se optimizan sus parámetros. Por otro lado, el bajo valor de la estadística KS y el p-valor alto del Marsenne Twister demuestran que este generador produce números pseudoaleatorios con una distribución uniforme aceptable. Esto confirma su idoneidad para simulaciones que requieren alta calidad estadística.

4. Estudio de Caso: Simulación de Actividades de la Caja de Varga

Las actividades de la Caja de Varga son una serie de experiencias pedagógicas diseñadas para facilitar la enseñanza de conceptos matemáticos a través de la manipulación física y la exploración. El Viaje del Taxista y el Método de Urna son dos ejemplos destacados que permiten a los estudiantes interactuar con la probabilidad y la estadística de forma tangible.

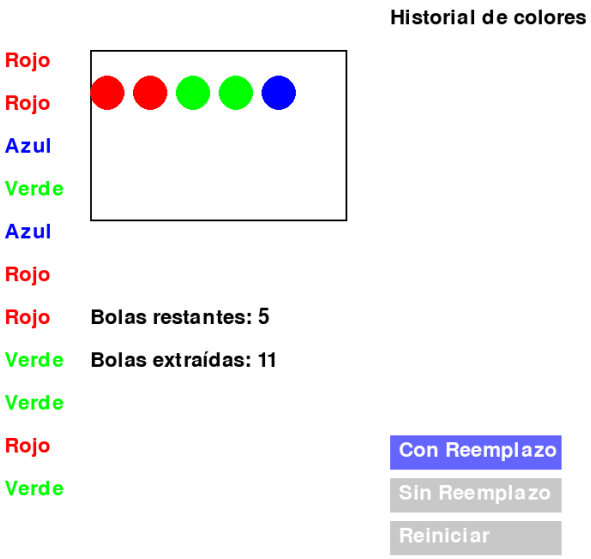
Como prácticas iniciales hemos implementado las dos situaciones.

En el Viaje del Taxista, saliendo desde el hotel, cada jugador lanza un dado con 3 caras verdes y 3 blancas. En el caso de que el resultado sea verde, el peón se desplaza a la derecha y en caso contrario a la izquierda. El propósito del juego es intentar llegar a uno de los dos avetos. La probabilidad de llegar a uno de los dos avetos con el mínimo de lanzamientos posible es de $\left(\frac{1}{2}\right)^{27}$ que es muy pequeña. Otra de las opciones que ofrece este juego es la variación de la casilla final para que los alumnos estudien la probabilidad de terminar en la iglesia, la estación de tren, la torre de observación, la fuente, la panadería o el aeropuerto.



Se puede consultar el código de esta simulación en el apartado B del anexo.

Por otro lado, el Método de Urna ilustra principios de la teoría de probabilidades, como el conteo de eventos favorables sobre el total de eventos posibles. Al simular la extracción de bolas de una urna virtual configurable, los estudiantes pueden observar cómo la variabilidad influye en los resultados.



Esta interacción directa con el concepto de probabilidad permite a los estudiantes entender la importancia del tamaño de la muestra y cómo las extracciones pueden aproximarse a la distribución teórica a medida que se repiten.

Se puede consultar el código en el apartado C del anexo.

5. El Problema de los Desarreglos y su Aplicación a La Caja de Varga

Una vez realizadas las simulaciones de las actividades del Viaje del taxista y el Método de la urna, pasamos ya a simular la caja de bolas correspondiente al problema de los desarreglos, introduciendo todo el estudio probabilístico en el que se basa.

El problema de los desarreglos, también conocido como el "problema de las coincidencias", tiene un trasfondo histórico interesante. Fue estudiado por primera vez en 1708 por Pierre Rémond De Montmort y ha evolucionado con aportaciones de matemáticos como Euler, Laplace y Bernoulli. Inicialmente, este problema trata de calcular el número de formas en las que un conjunto de elementos puede ser reordenado de tal manera que ningún elemento quede en su posición original. Posteriormente, se estudian los desarreglos parciales (Cultura Científica, 2017).

Este concepto es clave para resolver ciertos problemas de probabilidad, como el de la Caja de Varga, en la cual, encontramos una caja con 6 bolas de colores fijas y un compartimiento más grande con las mismas seis bolas. Al remover la caja las bolas del segundo compartimento se reordenan y comparamos dicha reordenación con las del compartimento de las bolas fijas. El objetivo es calcular la probabilidad de que ninguna bola quede en su posición original o que coincidan en una o más posiciones.

5.1. Definición del Problema

Dado un conjunto de n elementos, un desarreglo es una permutación en la que ningún elemento queda en su posición original. Matemáticamente, el número de desarreglos de un conjunto de n elementos se denota como dn o !n, también llamado subfactorial de n. La fórmula de recurrencia para calcular los desarreglos de n elementos es:

$$!n = (n-1) \cdot (!(n-1)+!(n-2)) \quad \forall \quad n \geq 2$$

5.2. Demostración de la fórmula por inducción

$$!0 = 1$$

$$!1 = 0$$

Vamos a ver que funciona para n = 2

$$!2 = (2-1) \cdot (!1+!0) = 1 \cdot (0+1) = 1$$

Que es correcto porque dados dos elementos solo hay una forma de desordenarlos.

Suponemos que se cumple para n - 2 y n - 1 y vamos a comprobar que también se cumple para n.

Como se trata de un desarreglo de n elementos, la primera posición la pueden ocupar n -1 elementos diferentes. Aquí nos podemos encontrar con dos casos.

1. Que el elemento 1 ocupe la posición correspondiente al primer elemento del desarreglo. Por lo tanto, nos queda un desarreglo de n - 2 posiciones que tendrá !(n - 2) opciones diferentes.

2. Que el elemento 1 no ocupe la posición correspondiente al primer elemento del desarreglo. Por lo tanto queda un desarreglo de n - 1 posiciones que tendrá !(n - 1) posiciones diferentes.

A partir de estos dos casos podemos deducir la fórmula general.

5.3. Cálculo de el número de desarreglos de n elementos

A continuación, se muestran los cálculos de los desarreglos para valores pequeños de n, utilizando la fórmula de recurrencia:

$$!0 = 1$$

$$!1 = 0$$

$$!2 = 1$$

$$!3 = 2$$

$$!4 = 9$$

$$!5 = 44$$

$$!6 = 265$$

Como ejemplo, para n=7, el número de desarreglos se calcula de la siguiente manera:

$$!7 = (7-1) \cdot (!6+!5) = 6 \cdot (265+44) = 1854$$

Este valor de !7=1854 indica que, si tuviéramos 7 elementos, habría 1854 formas diferentes de reordenarlos sin que ninguno quede en su posición original.

5.4. Aplicación: La Caja de Varga

En la actividad de la Caja de Varga, tenemos una caja con bolas de colores y el objetivo es calcular la probabilidad de que las bolas no coincidan en ninguna posición después de haber sido mezcladas. Para simplificar, consideremos el caso de 6 bolas de colores diferentes. La probabilidad de que ninguna bola quede en su posición original se puede calcular utilizando el número de desarreglos !6.

La probabilidad de que ninguna bola coincida con su posición original es:

$$P_6 = \frac{!6}{6!} = \frac{265}{720} \approx 0{,}3681$$

Esto significa que la probabilidad de que después de mezclar las bolas, ninguna quede en su posición original es de 0,3681. Por lo tanto, la probabilidad de que como mínimo haya 1 coincidencia es de 1 - 0,3681 = 0,6319.

En los estudios de Euler se puede comprobar como cuando el número de bolas utilizadas se hace grande $\lim_{n\to\infty} P_n(\text{No coinciden}) = \frac{1}{e}$
(Cuaderno de Cultura Científica, 2017).

5.5. Cálculo de Desarreglos Parciales en el caso n = 6 y sus probabilidades

Para los casos en que queremos calcular la probabilidad de que coincidan exactamente k bolas, utilizamos los desarreglos parciales (Cuaderno de Cultura Científica, 2017). Estos se denotan como R(n,k), donde n es el número total de bolas y k es el número de bolas que coinciden con su posición original. La fórmula para calcular los desarreglos parciales es:

$$R(n,k) = C_n^k \cdot !(n-k)$$

La fórmula se genera a partir del cálculo de todas las maneras de seleccionar las k bolas que coinciden C_n^k por todas las maneras posibles de desarreglos de las (n - k) bolas restantes. Si miramos el caso n = 6 podemos calcular los desarreglos parciales.

$$R(6,0) = C_0^6 \cdot !(6-0) = 1 \cdot 265 = 265$$

$$R(6,1) = C_1^6 \cdot !(6-1) = 6 \cdot 44 = 264$$

$$R(6,2) = C_2^6 \cdot !(6-2) = 15 \cdot 9 = 135$$

$$R(6,3) = C_3^6 \cdot !(6-3) = 20 \cdot 2 = 40$$

$$R(6,4) = C_4^6 \cdot !(6-4) = 15 \cdot 1 = 15$$

$$R(6,5) = C_5^6 \cdot !(6-5) = 6 \cdot 0 = 0$$

$$R(6,6) = C_6^6 \cdot !(6-6) = 1 \cdot 1 = 1$$

Comprobamos que la suma de todas los desarreglos parciales para n = 6 es igual a 720 que coincide con 6!, que son todas las formas posibles de reordenar 6 bolas.

Si ahora calculamos la probabilidad de los diferentes desarreglos parciales:

$$P(\text{No coincide ninguna bola}) = \frac{265}{720} \approx 0{,}3681$$

$$P(\text{Coincide exactamente 1 bola}) = \frac{264}{720} \approx 0{,}3667$$

$$P(\text{Coinciden exactamente 2 bolas}) = \frac{135}{720} \approx 0{,}1875$$

$$P(\text{Coinciden exactamente 3 bolas}) = \frac{40}{720} \approx 0{,}0556$$

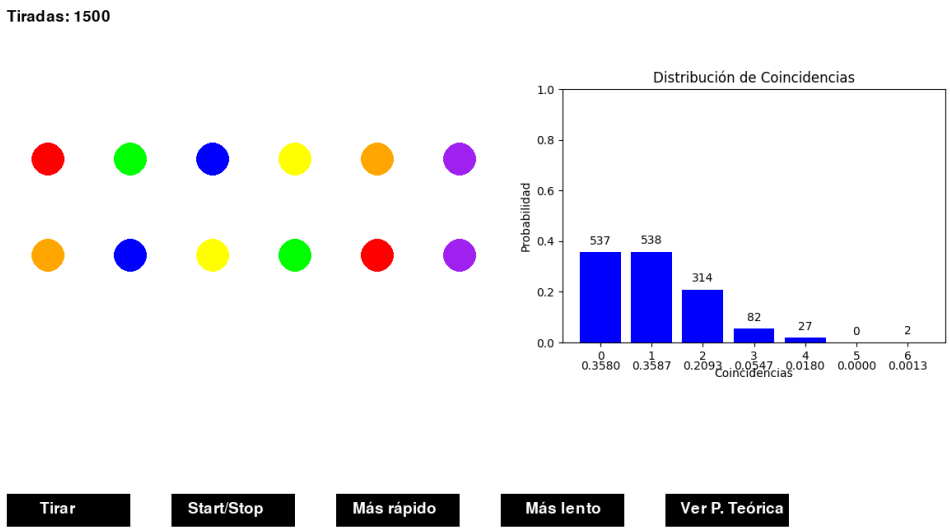
$$P(\text{Coinciden exactamente 4 bolas}) = \frac{15}{720} \approx 0{,}0208$$

$$P(\text{Coinciden exactamente 5 bolas}) = \frac{0}{720} = 0$$

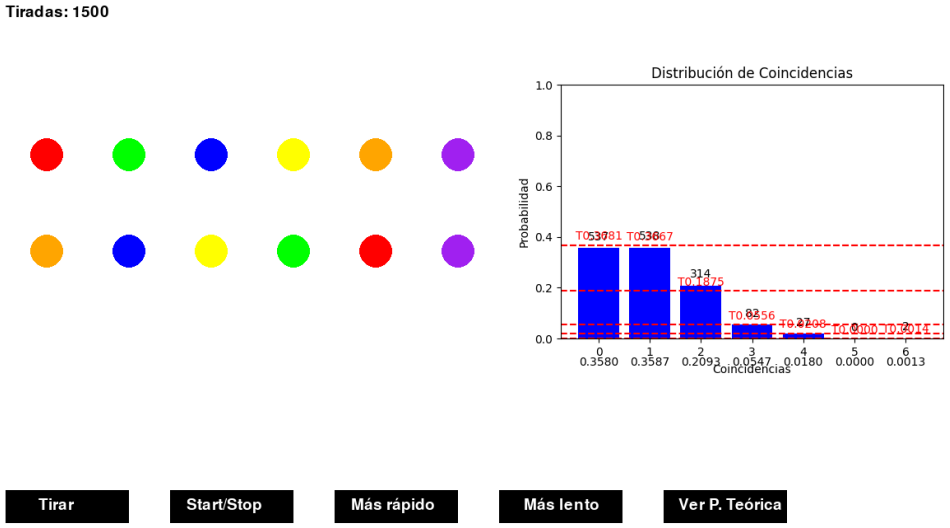
$$P(\text{Coinciden exactamente 6 bolas}) = \frac{1}{720} \approx 0{,}0014$$

5.6. Simulación del juego en Pygame

Para comprobar la eficiencia de los números pseudoaleatorios en este problema se ha generado un programa utilizando el algoritmo de Mersenne Twister, predeterminado en Python por la librería random. Este permite la actualización de los datos en un gráfico dinámico donde se ven los resultados de la experimentación simulada.

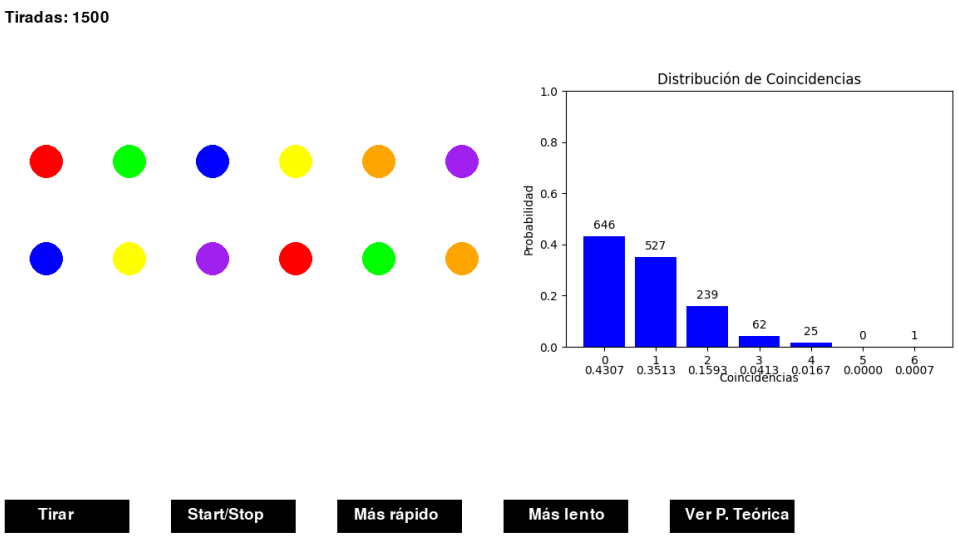


Los resultados de la simulación se acercan con bastante precisión a las probabilidades calculadas anteriormente, comparativa que también se pueden consultar en el gráfico como se muestra a continuación.

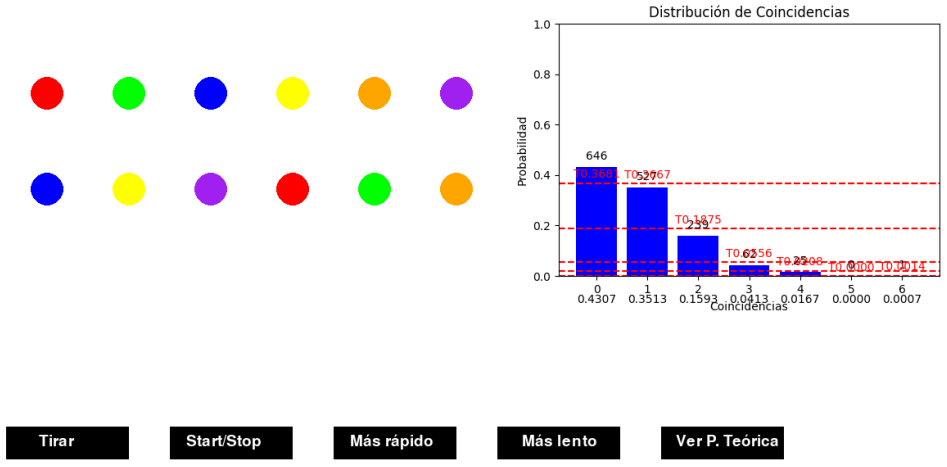


Se puede consultar el código en el apartado D del anexo.

Para poder hacer una comparación precisa se ha implementado el mismo código pero esta vez usando el algoritmo LGC, aportando los resultados que se muestran a continuación:



Como se ha hecho anteriormente si escogemos la opción de comparar con las probabilidades teóricas, los resultados de la simulación en este caso se acercan con significativamente menos precisión que en el Marsenne Twister.



Se puede consultar el código en el apartado E del anexo.

Además, se ha experimentado con los parámetros de LGC distinto al estándar usado para las simulaciones. En este se ha comprobado que si el módulo es pequeño, comprobado entre 10 y 100, y junto con el valor de c no son coprimos se genera un patrón que alterna entre dos de los valores posibles como podrían ser 1 y 2. Esta experimentación destaca la importancia de la selección adecuada de los parámetros y otra limitación que presenta este algoritmo, como se indicaba en la teoría.

6. Conclusiones

Los objetivos iniciales del trabajo eran, por una parte, entender los métodos matemáticos detrás de la generación de números pseudoaleatorios en ordenadores y utilizar este conocimiento para crear una simulación digital de algunos de los materiales y juegos que forman parte de la Caja de Varga.

Inicialmente, se simularon dos actividades propuestas en la "Caja de Varga" qué sirvieron como prácticas, para el desarrollo a posteriori más detallado del Problema de los Desarreglos, elegido porque permitía hacer un estudio teórico de las probabilidades más interesante que los anteriores.

Más adelante, se han revisado los métodos de generación de números pseudoaleatorios como son el Mersenne Twister y el LCG adecuados para hacer estas simulaciones.

A partir de este análisis se ha podido valorar la hipótesis inicial que confirma la teoría expuesta, argumentando con la prueba estadística de Kolmogorov-Smirnov y los resultados de las simulaciones. Se ha observado que el algoritmo del Marsenne Twister genera una precisión mucho más elevada concorde a los cálculos teóricos. Sin embargo, aunque el LGC genera un error más elevado respecto a la probabilidad teórica, se acerca lo suficiente como para ser considerado adecuado en el contexto de la simulación, debido a que la precisión requerida por los alumnos para entender los conceptos no es tan crucial. Además este segundo algoritmo tiene una estructura mucho más simple que el anterior y permite entender las limitaciones de los generadores de números pseudoaleatorios a través de la experimentación con sus parámetros, concepto que puede aportar conocimiento y reflexión al alumno.

La realización de este trabajo ha generado múltiples futuras opciones de investigación como por ejemplo el estudio de generación de números realmente aleatorios, la simulación completa de manera más sofisticada de la Caja de Varga, la utilización de esta herramienta con alumnos en el aula y la valoración de los profesores sobre el uso de estos métodos. También se puede considerar un estudio con más profundidad añadiendo tests para hacer una valoración más rígida y la comparación con otros algoritmos para valorar el alcance que podrían llegar a tener los dos algoritmos estudiados.

Referencias

[1] Chaparro, L. (2020). A brief history of randomness. *ResearchGate*. https://www.researchgate.net/publication/340515665_A_brief_history_of_randomness

[2] Cultura Científica. (2017). "Problema Matemático: Las Cartas Extraviadas."20 de septiembre de 2017. https://culturacientifica.com/2017/09/20/problema-matematico-las-cartas-extraviadas/

[3] Knuth, D. E. (1969). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (2nd ed.). Addison-Wesley.

[4] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.

[5] Kolmogorov, A. N. (1933). *Grundbegriffe der Wahrscheinlichkeitsrechnung (Fundamentos de la teoría de la probabilidad)*. Springer-Verlag.

[6] Matsumoto, M., & Nishimura, T. (1998). Mersenne Twister: A 623-dimensionally equidistributed uniform pseudorandom number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS), 8(1)*, 3-30.

Anexo

A. Test de Kolmogórov-Smirnov

```
import numpy as np
from scipy.stats import ks_1samp, uniform

def lcg(modulus, a, c, seed, size):
    x = seed
    numbers = []
    for _ in range(size):
        x = (a * x + c) % modulus
        numbers.append(x / modulus)
    return np.array(numbers)

modulus = 2**31 - 1
a = 1103515245
c = 12345
seed = 42

size = 10000
lcg_numbers = lcg(modulus, a, c, seed, size)
mersenne_numbers = np.random.random(size)  # Usando el Mersenne
Twister predeterminado de NumPy

lcg_ks_stat, lcg_p_value = ks_1samp(lcg_numbers, uniform.cdf)
mersenne_ks_stat, mersenne_p_value = ks_1samp(mersenne_numbers,
uniform.cdf)

print("LCG:")
print(f"Estadística KS: {lcg_ks_stat:.4f}, p-valor:
{lcg_p_value:.4f}")

print("\nMersenne Twister:")
print(f"Estadística KS: {mersenne_ks_stat:.4f}, p-valor:
{mersenne_p_value:.4f}")
```

B. Código del Viaje del Taxista

```
import pygame
import random
pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("El viaje casual del taxista")

tablero = pygame.image.load("tablero.png")
```

```
tablero = pygame.transform.scale(tablero, (WIDTH, HEIGHT))

RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
YELLOW = (255, 255, 0)
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

casillas = [
    (251, 367), (167, 325), (203, 276), (268, 278), (306, 317), (305, 391),
    (244, 426), (178, 426), (122, 365), (121, 288), (173, 227), (234, 231),
    (303, 231), (352, 286), (350, 353), (351, 422), (308, 476), (246, 475),
    (180, 474), (115, 474), (74, 433), (75, 363), (77, 294), (76, 222),
    (139, 180), (222, 181), (310, 181), (400, 182), (486, 184), (571, 183),
    (658, 185), (716, 227), (715, 296), (715, 363), (716, 435), (677, 478),
    (608, 477), (544, 478), (478, 477), (440, 423), (440, 354), (439, 286),
    (487, 232), (558, 233), (619, 230), (672, 294), (670, 367), (607, 430),
    (545, 427), (486, 390), (487, 322), (523, 282), (586, 282), (626, 329),
    (541, 360)
]

posiciones_finales = [0, len(casillas) - 1]

num_peons = 0
peons = []
peon_colors = []
PEON_RADIUS = 10
font = pygame.font.Font(None, 36)

def lanzar_dado():
    return random.choice(["green", "white"])

def mover_peon(peon_index, resultado):
    if resultado == "green":
        if peons[peon_index] < len(casillas) - 1:
            peons[peon_index] += 1
    else:
        if peons[peon_index] > 0:
            peons[peon_index] -= 1

def verificar_ganador():
    for i, peon_pos in enumerate(peons):
        if peon_pos in posiciones_finales:
            return i + 1
    return None

def pantalla_seleccion_jugadores():
```

```
    global num_peons, peons, peon_colors

    seleccionando = True
    while seleccionando:
        screen.fill(WHITE)
        titulo_texto = font.render("Selecciona el número de jugadores",
        True, BLACK)
        screen.blit(titulo_texto, (WIDTH // 2 -
        titulo_texto.get_width() // 2, HEIGHT // 4))

        botones = []
        for i in range(1, 5):
            boton_rect = pygame.Rect(WIDTH // 2 - 50, HEIGHT // 2 + (i
            - 2) * 60, 100, 50)
            botones.append(boton_rect)
            pygame.draw.rect(screen, BLACK, boton_rect, 2)
            texto_boton = font.render(f"{i}", True, BLACK)
            screen.blit(texto_boton, (boton_rect.x + boton_rect.width
            // 2 - texto_boton.get_width() // 2,
                                      boton_rect.y + boton_rect.height
                                      // 2 - texto_boton.get_height() // 2))


        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                exit()


            if event.type == pygame.MOUSEBUTTONDOWN:
                for i, boton in enumerate(botones):
                    if boton.collidepoint(event.pos):
                        num_peons = i + 1
                        peons = [len(casillas) // 2] * num_peons
                        peon_colors = [RED, GREEN, BLUE, YELLOW][:num_peons]
                        seleccionando = False


        pygame.display.flip()

pantalla_seleccion_jugadores()

running = True
turn = 0
resultado_dado = ""
ganador = None
clock = pygame.time.Clock()
```

```
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False


        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE and ganador is None:
                resultado_dado = lanzar_dado()
                print(f"Resultado del dado: {resultado_dado}")


                mover_peon(turn, resultado_dado)

                ganador = verificar_ganador()
                if ganador:
                    print(f"¡El jugador {ganador} ha ganado!")
                if not ganador:
                    turn = (turn + 1) % num_peons

    screen.blit(tablero, (0, 0))

    for i, peon_pos in enumerate(peons):
        pos = casillas[peon_pos]
        pygame.draw.circle(screen, peon_colors[i], pos, PEON_RADIUS)


    dado_pos = (WIDTH // 2 - 25, HEIGHT - 100)
    pygame.draw.rect(screen, BLACK, (dado_pos[0], dado_pos[1], 50, 50),
    2)

    if resultado_dado == "green":
        pygame.draw.circle(screen, GREEN, (dado_pos[0] + 25,
        dado_pos[1] + 25), 20)
        pygame.draw.circle(screen, BLACK, (dado_pos[0] + 25,
        dado_pos[1] + 25), 20, 2)
    elif resultado_dado == "white":
        pygame.draw.circle(screen, WHITE, (dado_pos[0] + 25,
        dado_pos[1] + 25), 20)
        pygame.draw.circle(screen, BLACK, (dado_pos[0] + 25,
        dado_pos[1] + 25), 20, 2)

    if ganador:
        ganador_texto = font.render(f"¡El jugador {ganador}
        ha ganado!", True, BLACK)
        screen.blit(ganador_texto, (WIDTH // 2 -
        ganador_texto.get_width() // 2, HEIGHT // 2))
```

```
        pygame.display.flip()


        clock.tick(30)

pygame.quit()

print("Gracias por jugar.")
```

C. Código del Método de Urnas

```
import pygame
import random

pygame.init()

screen_width, screen_height = 800, 700
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("Modelo de Urna Personalizable")

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GRAY = (200, 200, 200)
BLUE = (100, 100, 255)
RED = (255, 100, 100)

color_nombres = {
    (255, 0, 0): "Rojo",
    (0, 255, 0): "Verde",
    (0, 0, 255): "Azul",
    (255, 255, 0): "Amarillo",
    (0, 255, 255): "Cian",
    (255, 0, 255): "Magenta"
}

font = pygame.font.SysFont(None, 36)

config_urnas = {
    "Rojo": 3,
    "Verde": 3,
    "Azul": 3,
    "Amarillo": 2,
    "Cian": 2,
    "Magenta": 2,
}

bolas_en_urna = []
extraidas = []
simulacion_iniciada = False
modo_con_reemplazo = True


def generar_urna_config():
    global bolas_en_urna
    bolas_en_urna = []
    bolas_en_urna += [(255, 0, 0)] * config_urnas["Rojo"]
    bolas_en_urna += [(0, 255, 0)] * config_urnas["Verde"]
    bolas_en_urna += [(0, 0, 255)] * config_urnas["Azul"]
```

```
    bolas_en_urna += [(255, 255, 0)] * config_urnas["Amarillo"]
    bolas_en_urna += [(0, 255, 255)] * config_urnas["Cian"]
    bolas_en_urna += [(255, 0, 255)] * config_urnas["Magenta"]
    extraidas.clear()

def draw_text(text, x, y, color=BLACK):
    img = font.render(text, True, color)
    screen.blit(img, (x, y))

def draw_button(text, rect, active):
    color = BLUE if active else GRAY
    pygame.draw.rect(screen, color, rect)
    draw_text(text, rect.x + 10, rect.y + 5, WHITE)

def extraer_bola():
    if bolas_en_urna:
        bola = random.choice(bolas_en_urna)
        extraidas.append(bola)
        if not modo_con_reemplazo:
            bolas_en_urna.remove(bola)

def reiniciar_experimento():
    generar_urna_config()

def draw_config():
    y_start = 100
    for i, (color, cantidad) in enumerate(config_urnas.items()):
        draw_text(f"{color}: {cantidad}", 100, y_start + i * 60)

        rect_mas = pygame.Rect(300, y_start + i * 60, 50, 40)
        rect_menos = pygame.Rect(360, y_start + i * 60, 50, 40)
        draw_button("+", rect_mas, True)
        draw_button("-", rect_menos, True)
        config_buttons[color] = (rect_mas, rect_menos)

def draw_simulacion():
    screen.fill(WHITE)

    draw_urna()

    for i, color in enumerate(extraidas):
        nombre_color = color_nombres.get(color, "Desconocido")
        draw_text(f"{nombre_color}", 50, 100 + i * 50, color)

    draw_text("Historial de colores extraídos:", 500, 50)
    draw_text(f"Bolas restantes: {len(bolas_en_urna)}", 150, 400)
    draw_text(f"Bolas extraídas: {len(extraidas)}", 150, 450)
    draw_button("Con Reemplazo", rect_con_reemplazo,
```

```
    modo_con_reemplazo)
    draw_button("Sin Reemplazo", rect_sin_reemplazo, not
    modo_con_reemplazo)
    draw_button("Reiniciar", rect_reset, False)

def draw_urna():
    pygame.draw.rect(screen, BLACK, urna_rect, 2)
    for i, color in enumerate(bolas_en_urna):
        x = 170 + (i * 50) % 280
        y = 150 + (i // 5) * 50
        pygame.draw.circle(screen, color, (x, y), 20)

urna_rect = pygame.Rect(150, 100, 300, 200)
config_buttons = {}

rect_con_reemplazo = pygame.Rect(500, 550, 200, 40)
rect_sin_reemplazo = pygame.Rect(500, 600, 200, 40)
rect_reset = pygame.Rect(500, 650, 200, 40)

running = True
while running:
    screen.fill(WHITE)
    if simulacion_iniciada:

        draw_simulacion()
    else:

        draw_text("Configura la Urna:", 300, 50)
        draw_config()

        rect_iniciar = pygame.Rect(300, 600, 200, 50)
        draw_button("Iniciar Simulación", rect_iniciar, True)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if not simulacion_iniciada:

                for color, (rect_mas, rect_menos) in
                config_buttons.items():
                    if rect_mas.collidepoint(event.pos):
                        config_urnas[color] += 1
                    elif rect_menos.collidepoint(event.pos) and
                    config_urnas[color] > 0:
                        config_urnas[color] -= 1

                if rect_iniciar.collidepoint(event.pos):
```

```
                    generar_urna_config()
                    simulacion_iniciada = True
            else:

                if rect_con_reemplazo.collidepoint(event.pos):
                    modo_con_reemplazo = True
                elif rect_sin_reemplazo.collidepoint(event.pos):
                    modo_con_reemplazo = False
                elif rect_reset.collidepoint(event.pos):
                    reiniciar_experimento()

                extraer_bola()


    pygame.display.flip()

pygame.quit()
```

D. Códido de simulación de desarreglos usando Marsenne Twister

```
import pygame
import random
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.figure import Figure
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
import numpy as np

matplotlib.use("Agg")

pygame.init()

WIDTH, HEIGHT = 1200, 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Simulación de Bolas con Probabilidad")

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
YELLOW = (255, 255, 0)
ORANGE = (255, 165, 0)
PURPLE = (160, 32, 240)
COLORS = [RED, GREEN, BLUE, YELLOW, ORANGE, PURPLE]

BOLA_RADIUS = 20
user_balls = [(100 + i * 100, HEIGHT // 3) for i in range(6)]
```

```
random_balls = [(100 + i * 100, HEIGHT // 2) for i in range(6)]

button_width, button_height = 150, 40
button_roll = pygame.Rect(50, HEIGHT - 60, button_width, button_height)
button_start_stop = pygame.Rect(250, HEIGHT - 60, button_width, button_height)
button_speed_up = pygame.Rect(450, HEIGHT - 60, button_width, button_height)
button_speed_down = pygame.Rect(650, HEIGHT - 60, button_width, button_height)
button_theory = pygame.Rect(850, HEIGHT - 60, button_width, button_height)

user_order = list(range(6))
random_order = []
match_counts = [0] * 7
total_rolls = 0
rolling = False
delay = 500
graph_surface = None
show_theory = False

theoretical_probs = [0.3681, 0.3667, 0.1875, 0.0556, 0.0208, 0.0, 0.0014]

font = pygame.font.SysFont(None, 28)
small_font = pygame.font.SysFont(None, 24)
clock = pygame.time.Clock()

def draw_balls(positions, order, colors):
    for i, pos in enumerate(positions):
        pygame.draw.circle(screen, colors[order[i]], pos, BOLA_RADIUS)

def calculate_match(user_order, random_order):
    matches = sum([1 for i in range(6) if user_order[i] == random_order[i]])
    return matches

def draw_graph():
    global graph_surface
    fig = Figure(figsize=(6, 4), dpi=100)
    ax = fig.add_subplot(111)
    x = np.arange(7)
    probabilities = [count / total_rolls if total_rolls > 0 else 0
    for count in match_counts]

    bars = ax.bar(x, probabilities, color='blue')
    ax.set_ylim(0, 1)
    ax.set_xticks(x)
    ax.set_xticklabels([f'{i}' for i in range(7)], rotation=0, ha="center")
    ax.set_xlabel('Coincidencias', labelpad=5)
    ax.set_ylabel('Probabilidad')
    ax.set_title('Distribución de Coincidencias')
```

```
    for bar, count, prob in zip(bars, match_counts, probabilities):
        yval = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
        f'{count}', ha='center', va='bottom', fontsize=10)
        ax.text(bar.get_x() + bar.get_width()/2, -0.07,
        f'{prob:.4f}', ha='center', va='top', fontsize=10)

    if show_theory:
        for i, theory_prob in enumerate(theoretical_probs):
            ax.axhline(y=theory_prob, color='red', linestyle='--')
            ax.text(i, theory_prob + 0.02, f'T{theory_prob:.4f}',
            color='red', ha='center', fontsize=10)

    canvas = FigureCanvas(fig)
    canvas.draw()
    renderer = canvas.get_renderer()
    raw_data = renderer.buffer_rgba()
    plt.close(fig)

    graph_surface = pygame.image.frombuffer(raw_data,
    canvas.get_width_height(), "RGBA")
    return graph_surface

def handle_events():
    global rolling, delay, random_order, total_rolls, match_counts,
    graph_surface, show_theory

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.MOUSEBUTTONDOWN:
            if button_roll.collidepoint(event.pos):
                random_order = random.sample(range(6), 6)
                matches = calculate_match(user_order, random_order)
                match_counts[matches] += 1
                total_rolls += 1
                graph_surface = draw_graph()

            elif button_start_stop.collidepoint(event.pos):
                rolling = not rolling

            elif button_speed_up.collidepoint(event.pos):
                delay = max(100, delay - 10000)

            elif button_speed_down.collidepoint(event.pos):
                delay += 100
```

```
            elif button_theory.collidepoint(event.pos):
                show_theory = not show_theory
                graph_surface = draw_graph()


    return True

running = True
while running:

    screen.fill(WHITE)

    running = handle_events()

    if rolling:
        random_order = random.sample(range(6), 6)
        matches = calculate_match(user_order, random_order)
        match_counts[matches] += 1
        total_rolls += 1
        graph_surface = draw_graph()
        pygame.time.delay(delay)

    if len(random_order) == 6:
        draw_balls(random_balls, random_order, COLORS)
    if len(user_order) == 6:
        draw_balls(user_balls, user_order, COLORS)

    pygame.draw.rect(screen, BLACK, button_roll)
    pygame.draw.rect(screen, BLACK, button_start_stop)
    pygame.draw.rect(screen, BLACK, button_speed_up)
    pygame.draw.rect(screen, BLACK, button_speed_down)
    pygame.draw.rect(screen, BLACK, button_theory)

    screen.blit(font.render("Tirar", True, WHITE), (button_roll.x +
    40, button_roll.y + 8))
    screen.blit(font.render("Start/Stop", True, WHITE),
    (button_start_stop.x + 20, button_start_stop.y + 8))
    screen.blit(font.render("Más rápido", True, WHITE),
    (button_speed_up.x + 20, button_speed_up.y + 8))
    screen.blit(font.render("Más lento", True, WHITE),
    (button_speed_down.x + 30, button_speed_down.y + 8))
    screen.blit(font.render("Ver P. Teórica", True, WHITE),
    (button_theory.x + 18, button_theory.y + 8))

    screen.blit(font.render(f"Tiradas: {total_rolls}", True,
    BLACK), (50, 50))

    if graph_surface is not None:
```

```
        screen.blit(graph_surface, (650, 100))

    pygame.display.flip()

    clock.tick(30)
pygame.quit()
```

E. Código de simulación de desarreglos usando LGC

```
import pygame
import random
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.figure import Figure
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
import numpy as np

matplotlib.use("Agg")
pygame.init()
WIDTH, HEIGHT = 1200, 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Simulación de Desarreglos (LCG)")

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
YELLOW = (255, 255, 0)
ORANGE = (255, 165, 0)
PURPLE = (160, 32, 240)
COLORS = [RED, GREEN, BLUE, YELLOW, ORANGE, PURPLE]

BOLA_RADIUS = 20
user_balls = [(100 + i * 100, HEIGHT // 3) for i in range(6)]
random_balls = [(100 + i * 100, HEIGHT // 2) for i in range(6)]

def lcg(modulus, a, c, seed):
    while True:
        seed = (a * seed + c) % modulus
        yield seed / modulus  # Normalizado entre 0 y 1

modulus = 2**31 - 1
a = 1103515245
c = 12345
seed = 42
lcg_generator = lcg(modulus, a, c, seed)
```

```
def generar_desarreglo():
    lista = list(range(6))
    for i in range(len(lista)):
        j = int(next(lcg_generator) * len(lista))
        lista[i], lista[j] = lista[j], lista[i]
    return lista

button_width, button_height = 150, 40
button_roll = pygame.Rect(50, HEIGHT - 60, button_width,
button_height)
button_start_stop = pygame.Rect(250, HEIGHT - 60, button_width,
button_height)
button_speed_up = pygame.Rect(450, HEIGHT - 60, button_width,
button_height)
button_speed_down = pygame.Rect(650, HEIGHT - 60, button_width,
button_height)
button_theory = pygame.Rect(850, HEIGHT - 60, button_width,
button_height)

user_order = list(range(6))
random_order = []
match_counts = [0] * 7
total_rolls = 0
rolling = False
delay = 500
graph_surface = None
show_theory = False

theoretical_probs = [0.3681, 0.3667, 0.1875, 0.0556, 0.0208, 0.0,
0.0014]

font = pygame.font.SysFont(None, 28)
small_font = pygame.font.SysFont(None, 24)
clock = pygame.time.Clock()

def draw_balls(positions, order, colors):
    for i, pos in enumerate(positions):
        pygame.draw.circle(screen, colors[order[i]], pos,
        BOLA_RADIUS)

def calculate_match(user_order, random_order):
    matches = sum([1 for i in range(6) if user_order[i] ==
    random_order[i]])
    return matches

def draw_graph():
    global graph_surface
```

```
    fig = Figure(figsize=(6, 4), dpi=100)
    ax = fig.add_subplot(111)
    x = np.arange(7)
    probabilities = [count / total_rolls if total_rolls > 0 else 0
    for count in match_counts]
    bars = ax.bar(x, probabilities, color='blue')
    ax.set_ylim(0, 1)
    ax.set_xticks(x)
    ax.set_xticklabels([f'{i}' for i in range(7)], rotation=0,
    ha="center")
    ax.set_xlabel('Coincidencias', labelpad=5)
    ax.set_ylabel('Probabilidad')
    ax.set_title('Distribución de Coincidencias')
    for bar, count, prob in zip(bars, match_counts, probabilities):
        yval = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
        f'{count}', ha='center', va='bottom', fontsize=10)
        ax.text(bar.get_x() + bar.get_width()/2, -0.07,
        f'{prob:.4f}', ha='center', va='top', fontsize=10)
    if show_theory:
        for i, theory_prob in enumerate(theoretical_probs):
            ax.axhline(y=theory_prob, color='red', linestyle='--')
            ax.text(i, theory_prob + 0.02, f'T{theory_prob:.4f}',
            color='red', ha='center', fontsize=10)
    canvas = FigureCanvas(fig)
    canvas.draw()
    renderer = canvas.get_renderer()
    raw_data = renderer.buffer_rgba()
    plt.close(fig)
    graph_surface = pygame.image.frombuffer(raw_data,
    canvas.get_width_height(), "RGBA")
    return graph_surface

def handle_events():
    global rolling, delay, random_order, total_rolls,
    match_counts, graph_surface, show_theory
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False
        if event.type == pygame.MOUSEBUTTONDOWN:
            if button_roll.collidepoint(event.pos):
                random_order = generar_desarreglo()
                matches = calculate_match(user_order, random_order)
                match_counts[matches] += 1
                total_rolls += 1
                graph_surface = draw_graph()
            elif button_start_stop.collidepoint(event.pos):
                rolling = not rolling
```

```
            elif button_speed_up.collidepoint(event.pos):
                delay = max(100, delay - 100)
            elif button_speed_down.collidepoint(event.pos):
                delay += 100
            elif button_theory.collidepoint(event.pos):
                show_theory = not show_theory
                graph_surface = draw_graph()
    return True

running = True
while running:
    screen.fill(WHITE)
    running = handle_events()
    if rolling:
        random_order = generar_desarreglo()
        matches = calculate_match(user_order, random_order)
        match_counts[matches] += 1
        total_rolls += 1
        graph_surface = draw_graph()
        pygame.time.delay(delay)
    if len(random_order) == 6:
        draw_balls(random_balls, random_order, COLORS)
    if len(user_order) == 6:
        draw_balls(user_balls, user_order, COLORS)
    pygame.draw.rect(screen, BLACK, button_roll)
    pygame.draw.rect(screen, BLACK, button_start_stop)
    pygame.draw.rect(screen, BLACK, button_speed_up)
    pygame.draw.rect(screen, BLACK, button_speed_down)
    pygame.draw.rect(screen, BLACK, button_theory)
    screen.blit(font.render("Tirar", True, WHITE), (button_roll.x
    + 40, button_roll.y + 8))
    screen.blit(font.render("Start/Stop", True, WHITE),
    (button_start_stop.x + 20, button_start_stop.y + 8))
    screen.blit(font.render("Más rápido", True, WHITE),
    (button_speed_up.x + 20, button_speed_up.y + 8))
    screen.blit(font.render("Más lento", True, WHITE),
    (button_speed_down.x + 30, button_speed_down.y + 8))
    screen.blit(font.render("Ver P. Teórica", True, WHITE),
    (button_theory.x + 18, button_theory.y + 8))
    screen.blit(font.render(f"Tiradas: {total_rolls}", True,
    BLACK), (50, 50))
    if graph_surface is not None:
        screen.blit(graph_surface, (650, 100))
    pygame.display.flip()
    clock.tick(30)

pygame.quit()
```